Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

Звіт про виконання лабораторної роботи №1
З дисципліни **«Функціональне програмування»**
На тему **«Основи функціонального програмування в python»**
Варіант 9

Виконав:
Студент групи ФЕП-23
Литвинюк Ярослав

Перевірив:
проф. Свелеба С. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати typing.Callable для параметризації обчислень.

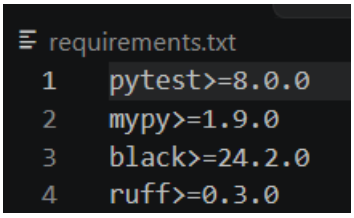
Завдання(варіант 9): Прокат автомобілів.

- **Вхідні дані:** Клас авто, дні, денна ставка, статус.
- **Основне завдання:** Обчислити базову ціну, знижку за тривалість і страховий збір.
- **Функціональні вимоги:** Політики знижки та страхування - функції.

Хід роботи

```
lab01/
├── requirements.txt
├── core.py
├── app.py
├── README.md
└── tests/
    └── test_core.py
```

requirements.txt



core.py

core.py > make_rental_processor > process

```python
"""
core.py — Чисте функціональне ядро для Варіанта №9: Прокат автомобілів.
Містить детерміновані чисті функції без побічних ефектів, глобальних змінних та мутацій.
"""

from collections.abc import Callable, Iterable
from copy import deepcopy
from typing import Any, TypedDict, TypeVar


class Rental(TypedDict, total=False):
    id: int
    car_class: str  # 'economy', 'standard', 'suv', 'premium'
    days: int
    daily_rate: float
    status: str  # 'confirmed', 'pending', 'cancelled'
    base_price: float
    discount_amount: float
    insurance_fee: float
    total: float


FilterPolicyFn = Callable[[Rental], bool]
DiscountPolicyFn = Callable[
    [float, int], float
]  # (base_price, days) -> discount_amount
InsurancePolicyFn = Callable[
    [float, str], float
]  # (discounted_subtotal, car_class) -> insurance_fee
NowFn = Callable[[], float]  # Інжекція часу без побічних ефектів

T = TypeVar("T")
U = TypeVar("U")
V = TypeVar("V")

def compose(f: Callable[[U], V], g: Callable[[T], U]) -> Callable[[T], V]:
    """Математична композиція двох функцій: f(g(x))."""
    return lambda x: f(g(x))


def make_multiplier(k: float) -> Callable[[float], float]:
    """Фабрика множників (замикання)."""
    return lambda x: round(x * k, 2)
```

```python
# --- Чисті функції предметної області (Варіант №9) ---


def calculate_base_price(rental: Rental) -> float:
    """Обчислює базову вартість оренди: дні * денна ставка."""
    return round(rental["days"] * rental["daily_rate"], 2)


def default_duration_discount(base_price: float, days: int) -> float:
    """
    Політика знижки за тривалістю:
    - від 7 днів: 15%
    - від 3 до 6 днів: 10%
    - менше 3 днів: 0%
    """
    if days >= 7:
        return round(base_price * 0.15, 2)
    if days >= 3:
        return round(base_price * 0.10, 2)
    return 0.0


def default_insurance_fee(subtotal: float, car_class: str) -> float:
    """
    Політика розрахунку обов'язкового страхового збору за класом авто:
    - premium: 15%
    - suv: 10%
    - economy / standard: 5%
    """
    rates = {
        "premium": 0.15,
        "suv": 0.10,
        "standard": 0.05,
        "economy": 0.05,
    }
    rate = rates.get(car_class.lower(), 0.05)
    return round(subtotal * rate, 2)
```

```python
def default_rental_filter(rental: Rental) -> bool:
    """Критерій валідності: статус 'confirmed' та тривалість більше 0 днів."""
    return rental.get("status") == "confirmed" and rental.get("days", 0) > 0


def with_rental_totals(
    rental: Rental,
    *,
    base_price: float,
    discount_amount: float,
    insurance_fee: float,
    total: float,
) -> Rental:
    """
    Повертає новий словник із розрахованими показниками,
    гарантуючи незмінність (immutability) вхідного об'єкта.
    """
    return {
        **rental,
        "base_price": base_price,
        "discount_amount": discount_amount,
        "insurance_fee": insurance_fee,
        "total": total,
    }


def stamp_record(record: Rental, now: NowFn) -> tuple[Rental, float]:
    """Приклад ізоляції залежності від часу через інжекцію Callable."""
    return deepcopy(record), now()
```

```python
# --- Функція вищого порядку (Фабрика обробника) ---


def make_rental_processor(
    *,
    filter_policy: FilterPolicyFn = default_rental_filter,
    discount_policy: DiscountPolicyFn = default_duration_discount,
    insurance_policy: InsurancePolicyFn = default_insurance_fee,
) -> Callable[[Iterable[Rental]], dict[str, Any]]:
    """
    Фабрика, що конструює чистий конвеєр обробки з інжектованими політиками.
    """

    def process(rentals: Iterable[Rental]) -> dict[str, Any]:
        processed_rentals: list[Rental] = []
        total_revenue = 0.0
        total_discount = 0.0
        total_insurance = 0.0
```

```
136             for r in rentals:
137                 if not filter_policy(r):
138                     continue
139
140                 base = calculate_base_price(r)
141                 discount = discount_policy(base, r["days"])
142                 subtotal = base - discount
143                 insurance = insurance_policy(subtotal, r["car_class"])
144                 final_total = round(subtotal + insurance, 2)
145
146                 updated = with_rental_totals(
147                     r,
148                     base_price=base,
149                     discount_amount=discount,
150                     insurance_fee=insurance,
151                     total=final_total,
152                 )
153                 processed_rentals.append(updated)
154
155                 total_revenue += final_total
156                 total_discount += discount
157                 total_insurance += insurance
158
159             return {
160                 "count": len(processed_rentals),
161                 "revenue": round(total_revenue, 2),
162                 "total_discounts": round(total_discount, 2),
163                 "total_insurance": round(total_insurance, 2),
164                 "rentals": processed_rentals,
165             }
166
167         return process
170 def process_rentals_pure(
171     rentals: Iterable[Rental],
172     *,
173     filter_policy: FilterPolicyFn = default_rental_filter,
174     discount_policy: DiscountPolicyFn = default_duration_discount,
175     insurance_policy: InsurancePolicyFn = default_insurance_fee,
176 ) -> dict[str, Any]:
177     """Прямий чистий виклик процесора замовлень."""
178     processor = make_rental_processor(
179         filter_policy=filter_policy,
180         discount_policy=discount_policy,
181         insurance_policy=insurance_policy,
182     )
183     return processor(rentals)
184
185
186 with_total = with_rental_totals
187
```

app.py

app.py > ...

```python
"""
app.py — Оболонка побічних ефектів (Imperative Shell).
Відповідає за консольний ввід/вивід (I/O), конфігурацію політик через Callable,
демонстрацію інжекції часу та виведення звітів.
"""

import time
from datetime import datetime
from typing import Any

from core import (
    Rental,
    make_rental_processor,
    process_rentals_pure,
    stamp_record,
)
```

```python
def render_report(
    result: dict[str, Any], title: str = "ЗВІТ СИСТЕМИ ПРОКАТУ АВТОМОБІЛІВ"
) -> None:
    """
    Імперативна функція вводу/виводу (I/O).
    Форматує та виводить результати обчислень у консоль у вигляді таблиці.
    Споживає виключно чистий словник-результат без внесення мутацій.
    """
    print("=" * 82)
    print(f"{title:^82}")
    print("=" * 82)
    print(
        f"{'ID':<4} | {'Клас авто':<10} | {'Днів':<5} | {'Ставка':<8} | "
        f"{'Базова':<9} | {'Знижка':<8} | {'Страховка':<10} | {'Разом':<9}"
    )
    print("-" * 82)
    rentals: list[Rental] = result.get("rentals", [])
    for r in rentals:
        print(
            f"{r['id']:<4} | "
            f"{r['car_class'].capitalize():<10} | "
            f"{r['days']:<5} | "
            f"${r['daily_rate']:<7.2f} | "
            f"${r.get('base_price', 0.0):<8.2f} | "
            f"-${r.get('discount_amount', 0.0):<7.2f} | "
            f"+${r.get('insurance_fee', 0.0):<9.2f} | "
            f"${r.get('total', 0.0):<8.2f}"
        )
    print("-" * 82)
    print(f"Кількість підтверджених замовлень: {result['count']}")
    print(f"Сумарна надана знижка:             -${result['total_discounts']:.2f}")
    print(f"Сумарний страховий збір:           +${result['total_insurance']:.2f}")
    print(f"Загальний виторг компанії:          ${result['revenue']:.2f}")
    print("=" * 82)
```

```python
def main() -> None:
    # 1. Вхідний набір даних згідно зі специфікацією Варіанта №9
    raw_rentals: list[Rental] = [
        {
            "id": 101,
            "car_class": "economy",
            "days": 2,
            "daily_rate": 30.0,
            "status": "confirmed",  # 2 дні (<3) -> знижка 0%, страховка 5%
        },
        {
            "id": 102,
            "car_class": "standard",
            "days": 5,
            "daily_rate": 50.0,
            "status": "confirmed",  # 5 днів (3..6) -> знижка 10%, страховка 5%
        },
        {
            "id": 103,
            "car_class": "suv",
            "days": 4,
            "daily_rate": 85.0,
            "status": "cancelled",  # скасовано -> відсіюється фільтром
        },
        {
            "id": 104,
            "car_class": "suv",
            "days": 10,
            "daily_rate": 90.0,
            "status": "confirmed",  # 10 днів (>=7) -> знижка 15%, страховка 10%
        },
        {
            "id": 105,
            "car_class": "premium",
            "days": 7,
            "daily_rate": 180.0,
            "status": "confirmed",  # 7 днів (>=7) -> знижка 15%, страховка 15%
        },
        {
            "id": 106,
            "car_class": "economy",
            "days": 0,
            "daily_rate": 25.0,
            "status": "confirmed",  # 0 днів -> відсіюється фільтром
        },
        {
            "id": 107,
            "car_class": "premium",
            "days": 3,
            "daily_rate": 200.0,
            "status": "pending",  # pending -> відсіюється стандартним фільтром
        },
    ]
```

```python
    # --- Демонстрація 1: Обробка за дефолтними політиками ядра ---
    default_report = process_rentals_pure(raw_rentals)
    render_report(default_report, title="СТАНДАРТНИЙ РОЗРАХУНОК (DEFAULT POLICIES)")

    # --- Демонстрація 2: Конфігурація кастомних політик через Callable ---
    # Приклад: промо-акція (Promo Weekend / VIP):
    # - Підвищена знижка за тривалість (25% від 7 днів, 15% від 3 днів)
    def promo_discount_policy(base_price: float, days: int) -> float:
        if days >= 7:
            return round(base_price * 0.25, 2)
        if days >= 3:
            return round(base_price * 0.15, 2)
        return 0.0

    # - Фіксована знижена страховка (7%) для всіх типів авто
    def flat_insurance_policy(subtotal: float, car_class: str) -> float:
        return round(subtotal * 0.07, 2)

    # - Розширений критерій валідності: допускаємо і 'confirmed', і 'pending'
    flexible_filter = (
        lambda r: r.get("status") in {"confirmed", "pending"} and r.get("days", 0) > 0
    )
```

```python
    promo_processor = make_rental_processor(
        filter_policy=flexible_filter,
        discount_policy=promo_discount_policy,
        insurance_policy=flat_insurance_policy,
    )
    promo_report = promo_processor(raw_rentals)
    print()
    render_report(promo_report, title="АКЦІЙНИЙ РОЗРАХУНОК (CUSTOM CALLABLE POLICIES)")

    # --- Демонстрація 3: Інжекція недетермінованого часу (stamp_record) ---
    if default_report["rentals"]:
        first_record: Rental = default_report["rentals"][0]
        stamped_record, timestamp = stamp_record(first_record, time.time)
        formatted_time = datetime.fromtimestamp(timestamp).strftime("%Y-%m-%d %H:%M:%S")

        print()
        print(f"[Audit Log] Аудит замовлення ID #{stamped_record['id']}:")
        print(f"            Час фіксації:    {formatted_time} (UNIX: {timestamp:.4f})")
        print(f"            Підсумкова сума: ${stamped_record['total']:.2f}")


if __name__ == "__main__":
    main()
```

test_core.py
tests/test_core.py

tests > test_core.py > ...

```python
"""
tests/test_core.py — Pytest-тести для перевірки чистоти функціонального ядра (Варіант №9).
"""

from copy import deepcopy

import pytest

from core import (
    Rental,
    calculate_base_price,
    compose,
    make_multiplier,
    make_rental_processor,
    process_rentals_pure,
    stamp_record,
    with_rental_totals,
)
@pytest.fixture
def sample_rentals() -> list[Rental]:
    return [
        {
            "id": 1,
            "car_class": "economy",
            "days": 2,
            "daily_rate": 30.0,
            "status": "confirmed",
        },
        {
            "id": 2,
            "car_class": "standard",
            "days": 5,
            "daily_rate": 50.0,
            "status": "confirmed",
        },
        {
            "id": 3,
            "car_class": "suv",
            "days": 4,
            "daily_rate": 85.0,
            "status": "cancelled",
        },
        {
            "id": 4,
            "car_class": "premium",
            "days": 7,
            "daily_rate": 180.0,
            "status": "confirmed",
        },
    ]
```

```python
def test_calculate_base_price(sample_rentals: list[Rental]) -> None:
    assert calculate_base_price(sample_rentals[0]) == 60.0
    assert calculate_base_price(sample_rentals[1]) == 250.0


def test_with_rental_totals_immutability(sample_rentals: list[Rental]) -> None:
    original = sample_rentals[0]
    updated = with_rental_totals(
        original,
        base_price=60.0,
        discount_amount=0.0,
        insurance_fee=3.0,
        total=63.0,
    )

    assert updated["total"] == 63.0
    assert updated["base_price"] == 60.0
    assert "total" not in original
    assert "base_price" not in original
    assert updated is not original
```

```python
def test_referential_transparency(sample_rentals: list[Rental]) -> None:
    # Той самий вхід гарантує ідентичний вихід без побічних ефектів
    r1 = process_rentals_pure(sample_rentals)
    r2 = process_rentals_pure(sample_rentals)

    assert r1 == r2
    assert r1["count"] == 3
    # ID 1: base 60.0, disc 0.0, ins 3.0 -> total 63.0
    # ID 2: base 250.0, disc 25.0, ins 11.25 -> total 236.25
    # ID 4: base 1260.0, disc 189.0, ins 160.65 -> total 1231.65
    # Загальний revenue: 63.0 + 236.25 + 1231.65 = 1530.90
    assert r1["revenue"] == 1530.90


def test_no_mutation(sample_rentals: list[Rental]) -> None:
    original = deepcopy(sample_rentals)
    process_rentals_pure(sample_rentals)

    # Вхідні дані абсолютно не мутуються
    assert sample_rentals == original
    for r in sample_rentals:
        assert "total" not in r
```

```python
def test_callable_policies(sample_rentals: list[Rental]) -> None:
    # Перевірка гнучкої заміни політик через Callable
    custom_discount = lambda base, days: round(base * 0.20, 2)
    custom_insurance = lambda subtotal, car_class: round(subtotal * 0.10, 2)
    filter_all = lambda r: r.get("status") == "confirmed"

    processor = make_rental_processor(
        filter_policy=filter_all,
        discount_policy=custom_discount,
        insurance_policy=custom_insurance,
    )
    res = processor(sample_rentals)

    assert res["count"] == 3
    # ID 1: base 60, disc 12, sub 48, ins 4.8 -> total 52.8
    assert res["rentals"][0]["total"] == 52.8
```

```python
def test_compose_and_multiplier() -> None:
    fee1 = make_multiplier(1.10)
    fee2 = make_multiplier(1.05)
    total_calc = compose(fee2, fee1)

    # 100 * 1.10 = 110.0 -> 110.0 * 1.05 = 115.5
    assert round(total_calc(100.0), 2) == 115.5


def test_stamp_record() -> None:
    fixed_time = 1700000000.0
    mock_now = lambda: fixed_time

    record: Rental = {
        "id": 1,
        "car_class": "economy",
        "days": 2,
        "daily_rate": 30.0,
    }
    stamped, ts = stamp_record(record, mock_now)

    assert ts == fixed_time
    assert stamped["id"] == record["id"]
    assert stamped is not record
```

README.md
README

README.md > abc # Лабораторна робота №1: Основи функціонального програмування у Python

```
# Лабораторна робота №1: Основи функціонального програмування у Python

## Варіант №9: Прокат автомобілів

### Мета роботи
- Опанувати концепції чистоти функцій (pure functions), незмінності стану (immutability) та референтної прозорості (referential transparency).
- Реалізувати патерн **Functional Core / Imperative Shell**: відокремити математичне обчислення вартості оренди від операцій виведення результатів та системного часу.
- Застосувати типізовані функції вищого порядку (`typing.Callable`) для гнучкого налаштування бізнес-політик: знижок за тривалість оренди, страхових зборів та фільтрації замовлень.

---

### Виконання вимог варіанта №9:
- **Вхідні дані:** Клас авто (`car_class`), дні (`days`), денна ставка (`daily_rate`), статус (`status`).
- **Основне завдання:** Розраховано базову ціну (`base_price`), ступеневу знижку за тривалість (`tiered_duration_discount`) та страховий збір (`insurance_policy`).
- **Функціональні вимоги:**
  - Усі політики знижки та страхування передаються як об'єкти першого класу через `Callable`.
  - Відсутні мутації вхідних даних — функція `with_total()` генерує новий словник без модифікації переданого.
  - Жодних глобальних змінних: параметри передаються безпосередньо або через функції вищого порядку.
  - Повне відокремлення операцій I/O (виведення `print`) у файлі `app.py`.

---

### Інструкції із запуску

#### 1. Активація середовища та встановлення залежностей
```bash
python -m venv .venv
source .venv/bin/activate  # Для Linux/macOS
# .venv\Scripts\activate   # Для Windows

pip install -r requirements.txt
```

висновок: Опановано концепції чистих функцій, референтної прозорості та архітектурного шаблону *Functional Core / Imperative Shell*. Завдяки винесенню бізнес-політик у typing.Callable та відмові від мутацій досягнуто повної детермінованості обчислень та надійної ізоляції I/O-операцій.
```